

COS214 PRACTICAL 2 - MIXED DESIGN PATTERNS	1
Team Members:	1
TASK 1	1
TASK 2	6
TASK 3	8
TASK 4	8
TASK 5	9
TASK 6	9

COS214 PRACTICAL 2 - MIXED DESIGN PATTERNS

Team Members:

- Nico Sibiya ()
- Tshegofatso Kungwane ()

TASK 1

1.1

Movement:

- Current attempt: Strategy
- Ideal pattern: State
- Reason: State's intent is to "allow an object to alter its behaviour when its internal state changes." The state of movement is the actual state of the object - depending on the mode of travel/movement, the behaviour of the object may change (being able to use some routes on foot vs being limited to certain routes because you're using different modes of transport, etc - this is the blueprint of the State pattern)

Routes:

- Current attempt: State
- Ideal pattern: Strategy
- Reason: Strategy's intent is to "define a family of algorithms, encapsulate them (i.e to hide each algorithm in its own class), and to make them interchangeable." The routes that the traveller will use will come from a pool of algorithms chosen priority (shortest, fastest, scenic, cheapest, ...). The route is picked by the traveller and that traveller can decide to change a route - based on several factors - even in the middle of a trip. This is allowed by the strategy pattern (interchanging the algorithm and using the algorithm in the "family pool")

Map:

- Current attempt: Composite
- Ideal pattern: Composite
- Reason: Composite's intent is to "compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly." The world map is a hierarchical structure where Regions can contain Locations (leaves) and other Regions (branches). Clients can call methods like print() on both Locations and Regions uniformly—treating them the same way regardless of whether they're a leaf or a composite node. This is exactly what Composite solves

Place features:

- Current attempt: Decorator
- Ideal pattern: Decorator
- Reason: Decorator's intent is to "attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality." Place features (Weather, Toll, Quest) need to stack on Locations in any combination. Decorators allow you to wrap a Location with multiple feature decorators dynamically at runtime in any order and combination, avoiding the subclass explosion

Biomes:

- Current attempt: Abstract Factory
- Ideal pattern: Abstract Factory
- Reason: Abstract Factory's intent is to "provide an interface for creating families of related or dependent objects without specifying the concrete classes." Each biome factory produces a complete, coherent family: DesertFactory creates DesertTerrain + DesertNPC + DesertObstacle together. This ensures that a Desert biome never mixes with Ocean or Forest objects - the factory locks families together, which is exactly what Abstract Factory is designed to do

1.2

Abstract factory:

- abstractFactory: BiomeFactory
- concreteFactory: DesertFactory, OceanFactory, ForestFactory
- abstractProduct: Terrain, NPC, Obstacle
- concreteProduct: DesertNPC, DesertTerrain, OceanTerrain, OceanNPC, ForestObstacle, CityNPC

Decorator:

- Component: Place
- concreteComponent: Location

- Decorator: PlaceDecorator
- concreteDecorator: weatherDecorator, tollDecorator, questDecorator

Composite:

- Component: Place
- Leaf: Location
- Composite: Region
- Client: mapUser

Strategy:

- Strategy: RouteStrategy
- concreteStrategy: shortestRouteStrategy, fastestRouteStrategy, scenicRouteStrategy, cheapestRouteStrategy
- Context: Trip (uses the strategy)

State:

- State: MoveState (TravelMode)
- Context: Traveller
- concreteState: onFoot, byBicycle, byCar, byAir

1.3

- 1) The abstract factory pattern only has one virtual method that needs to be implemented whereas the blueprint of the pattern is that the abstract factory class will have more than one factory method that will be implemented. This violates the design of the pattern. This could lead to the different biomes being mixed (and families are not supposed to be mixed). The coherence property of each family of the factory is violated
- 2) Objects being created but no destructor mentioned in the classes. This results in memory being leaked, segmentation faults have a higher chance of occurring during runtime, and dangling pointers aren't prevented and memory ends up being used without being emptied
- 3) The swapping of the names of the classes. An object could call the wrong implementation (because the wrong algorithm would have been implemented)
- 4) Having only one "base" class that knows about ALL the classes. Each pattern is supposed to have its own implementation and the presence of one main abstract class for all the patterns, this results in the tight coupling (that we aim to eradicate)
- 5) The decorator class doesn't show clear relationships between the components. At runtime this could result in the different features not compiling or the decorator not being able to be wrapped by (more than one) concreteDecorator
- 6) One or more patterns have missing or incorrectly connected participants. For example, Abstract Factory missing the product family hierarchies (Terrain, NPC, Obstacle all need

their own inheritance chains). There is a potential loss of polymorphism - which may negatively affect the class objects during runtime

- 7) Other abstract classes are not noted. The consequences are similar to the above
- 8) The diagram fails to annotate abstract classes and operations correctly. By UML convention, abstract class names and abstract method names must be written in italics. Because Figure 1 does not use italics, it is impossible to tell which methods must be overridden by subclasses. This leads to runtime errors where derived classes fail to implement required behavior

1.4 The class that knows everything about every other function = GameManager. GameManager knows about:

- Movement modes (Strategy/State implementation)
- Route algorithms (State/Strategy implementation)
- Map structure (Composite implementation)
- Place features (Decorator implementation)
- Biome factories (Abstract Factory implementation)

This defeats the decoupling goals. It becomes difficult to extend and maintain the algorithm. GameManager is just a coordinator that initializes these objects. Each object manages its own pattern, not the other way around

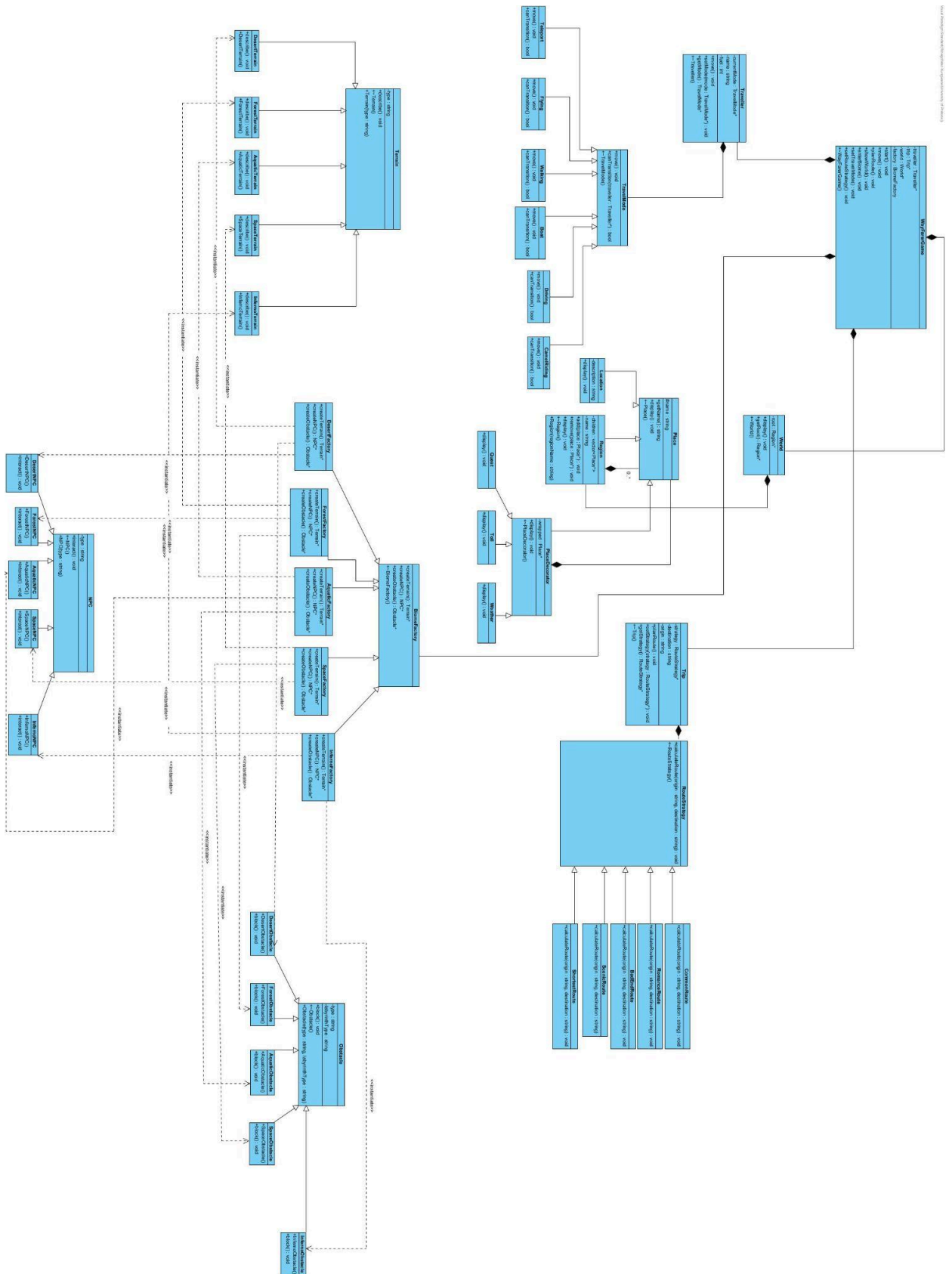
1.5 Figure 1's diagram structure is unclear and hard to follow. All subsystems and relationships converge on GameManager as a central point. Everything flows through one box, making it difficult to understand which class does what and who owns what. The five unrelated patterns are crammed together without clear separation or boundaries.

Readability:

- Can't see patterns isolated - All five patterns feed into GameManager, so you can't understand how each works independently.
- Relationships are unclear - The arrows don't show whether GameManager is creating, owning, or delegating to objects.
- Can't understand the intent - A developer can't see:
 - Why movement has guarded state transitions (Foot → Bicycle [has_bicycle] → Air [fuel>0])
 - Why routes are swappable algorithms (Shortest ↔ Fastest ↔ Scenic, anytime)
 - How the map builds hierarchically
- **No subsystem boundaries** - Each pattern is mixed together instead of clearly separated.

Conclusion: Each pattern should have its own clear diagram, then show high-level interactions between them. This way, each pattern's logic is visible and each object manages its own pattern—not GameManager controlling everything

TASK 2



State (Movement):

Traveller is the Context, TravelMode is the State. Concrete states are Walking, Bicycle, Car, Boat, Flying, and Transport. The Traveller owns the current mode via a raw pointer and deletes it manually in the destructor to ensure leak-free transitions.

Strategy (Routes):

Trip is the Context, RouteStrategy is the Strategy. Concrete strategies are ShortestPath, FastestPath, ScenicPath, and CheapestPath. The Trip owns its strategy via a raw pointer and deletes it in the destructor. Swapping strategies at runtime requires manual cleanup of the old strategy before assigning the new one.

Composite (Map):

Place is the Component. Location is the Leaf, Region is the Composite. The Region holds its children in a raw pointer vector and deletes them in its destructor. This allows the World root to delete the entire map tree recursively with one call.

Decorator (Place Features):

Place is the Component. PlaceDecorator is the abstract Decorator, and Quest, Toll, and Weather are concrete decorators. PlaceDecorator owns the wrapped Place and deletes it in its destructor. Deleting the outermost wrapper deletes the whole chain safely.

Abstract Factory (Biomes):

BiomeFactory is the abstract factory, with concrete factories DesertFactory, AquaticFactory, ForestFactory, InfernoFactory, and SpaceFactory. Abstract products are Terrain, NPC, and Obstacle. We used raw pointers and manual delete here to keep the interface simple, ensuring each family stays matched.

2.3

1. Transport mode (State):

A concrete state that moves the Traveller via public transit. Inherits from TravelMode and implements move(). The Traveller did not need to change.

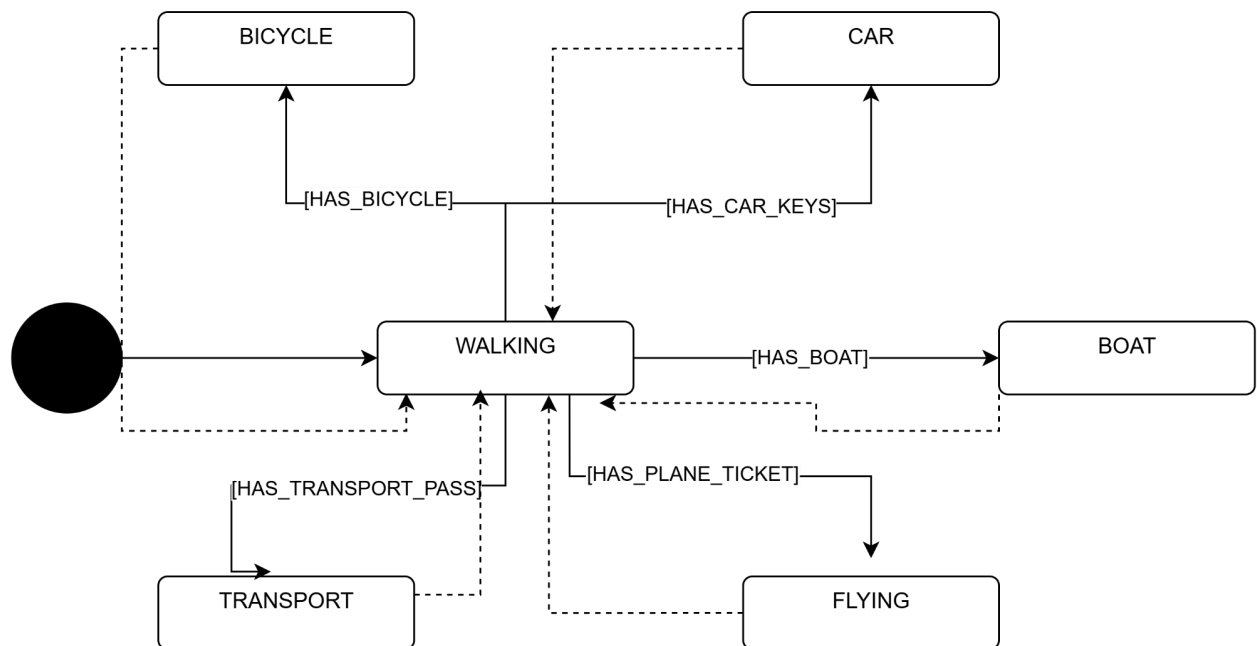
2. Space Biome (Abstract Factory):

SpaceTerrain, SpaceNPC, and SpaceObstacle form a complete space family. Added SpaceFactory that implements the abstract factory. No existing code was touched.

3. Inferno Biome (Abstract Factory):

A fire-themed family with InfernoTerrain, InfernoNPC, and InfernoObstacle. Added InfernoFactory that implements the same abstract interfaces. No existing code was changed.

TASK 3



TASK 4

4.3 Although the UML class diagrams for Traveller (State) and Trip (Strategy) appear structurally identical—both containing a pointer to an abstract base class—their intent is fundamentally different, making them non-interchangeable.

Movement is (a) State:

State's intent is to allow an object to alter its behaviour when its internal state changes. Movement is an integral part of what the Traveller is. The Traveller exists in a state of walking, driving, or flying (etc). These modes represent the actual condition of the Traveller object, not computational choices. When the Traveller transitions or changes between the modes, the behaviour of move() also changes because the Traveller's condition has changed. Mode selection is not algorithmic; it is determined by what the Traveller possesses and what transition guards permit. Guards enforce this because the state represents real, physical constraints.

Routes are Strategy:

Strategy's intent is to define a family of algorithms, encapsulate each one, and make them interchangeable. A route is a choice made by the Trip. The Trip does not become a "scenic trip"; it simply uses a scenic-route algorithm to calculate a path. The user selects which algorithm to use (Shortest, Fastest, Scenic), but the Trip object itself remains unchanged. The strategy is a tool that can be swapped without changing the Trip's identity.

Why they are not interchangeable:

Using Strategy for movement would imply that switching travel modes is merely swapping algorithms - which would miss the mark. One cannot transition to flying without meeting prerequisites like having a plane ticket.

Using State for routes would suggest the Trip assumes the identity of a specific route type. This is incorrect because routes are external calculations, and not internal conditions of the Trip.

Conclusion: State defines how an object behaves based on what it is. Strategy defines how an object behaves based on what tool it uses.

TASK 5

TASK 6

6.2

